

Masks Are Length Promises: How Diffusion-LM Agents Coordinate Parallel Tool Calls

Anonymous Author(s)

Submission Id: «OpenReview submission id»

“Can you check my loan eligibility for a home loan of amount \$500,000 from HSBC with annual income \$100,000 and for Wells Fargo for a amount of \$700,000 with annual income of \$120,000?”

Slot lengths	Call 1: institution, amount, income	Call 2: institution, amount, income	BFCL verdict	What went wrong
exact	HSBC, 500000, 100000	Wells Fargo, 700000, 120000	correct	—
one extra mask	HSBC Bank , 5000000, 1000000	Wells Fargo Bank , 7000000, 1200000	wrong	every value is lengthened; amounts ×10
swapped	Wells Fargo , 500000, 100000	HSBC , 700000, 120000	wrong	the banks are bound to the wrong calls

Figure 1: A dLLM agent’s parallel turn under three slot lengths. Dream fills the value slots of a fixed call skeleton committing one token per step, so no two decisions are ever made at once. With exact slot lengths the turn is correct. With one extra mask per slot, every value grows to fill its slot. With the two institutions’ slot lengths swapped, each call takes the bank whose name fits its slot: an error that looks like a coordination failure of parallel decoding, made without any parallel decoding.

Abstract

A tool-calling agent often issues several calls in one turn. Diffusion language models (dLLMs) commit many tokens per denoising step, which makes them attractive decision cores for such agents, and raises a coordination worry: tokens committed in the same step cannot see each other, so parallel calls may collide or bind each other’s arguments. We treat a dLLM agent’s parallel turn as simultaneous decisions by the value slots of a fixed call skeleton, and test what these decisions coordinate on, using the parallel calls of the Berkeley Function Calling Leaderboard. When the request orders its entities, the slots coordinate on mention order: committing 16 tokens per step instead of one costs Dream-7B 5 points of set accuracy, also where slot lengths cannot tell the calls apart. When nothing tells the calls apart, slots committed in the same step collide, in 100% of open choose-N requests against 0% one token at a time. Teams of LLM agents behave the same way: agents that act at once collide whether or not they are numbered, even when they sample, and agents that take turns never do. The costliest failures come from the action interface instead: the models read the number of masks in a slot as the length of its value. One extra mask per slot drops set accuracy from 90% to 16%, and swapping slot lengths between sibling calls makes 79% of the affected slots take the sibling value that fits, even one token per step. With estimated slot lengths, decoding one token per step makes about four times as many cross-call errors as decoding 16 tokens per step with exact lengths: errors an evaluator would blame on parallel decoding. A dLLM agent needs its slot lengths under control before parallelism becomes its problem.

Keywords

LLM agents, tool use, parallel action selection, coordination without communication, focal points, diffusion language models

ACM Reference Format:

Anonymous Author(s). 2027. Masks Are Length Promises: How Diffusion-LM Agents Coordinate Parallel Tool Calls. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 8 pages.

1 Introduction

[Planned: (1) Agents issue several tool calls per turn, a joint action; dLLMs commit many tokens per step and look like natural decision cores; the worry in coordination terms: same-step tokens are simultaneous moves without communication, so calls may collide or cross-bind. Lu et al. (ACL 2026) report that dLLM agents fail at tool calling and blame diffusion noise, without a controlled test. (2) The uncontrolled confound: a skeleton interface must fix slot lengths before decoding, and value lengths are unknown, so mismatch is the norm (fixed budgets → surplus; per-argument maximum → a sibling fits; estimates → both). (3) Our approach: a parallel turn as simultaneous decisions; predictions from coordination cues; independent control of the protocol (k , blocks) and the interface (slot length); a team-of-agents control. (4) Findings and contributions (structure.md §2). Figure 1: parallel_20 under exact / +1 / swap lengths, one token per step.]

2 Related Work

[Planned: LLM agents and parallel tool calls (LLMCompiler; BFCL parallel categories); dLLM agents (Lu et al. 2026; DLLM Agent; DLLM-Searcher; Agents of Diffusion, AAMAS 2026). Coordination without communication (Schelling focal points; Lewis conventions; dispersion games; zero-shot coordination, Other-Play): we apply these notions to the parallel decisions inside one agent and compare with a team of real agents. Parallel decoding and length in dLLMs, compressed: factorization error (Fast-dLLM, ParallelBench, EB-Sampler); variable length (CAL, single-span only; ρ -EOS; DAEDAL; S^3); constrained decoding (Dang & Ermon; Mündler et al.).]



This work is licensed under a Creative Commons Attribution International 4.0 License.

3 A dLLM Agent’s Parallel Turn as Simultaneous Decisions

3.1 The Agent and Its Action Interface

A tool-calling agent receives a request q and a set of tool schemas $\mathcal{F}$, and acts by issuing a set of calls $A = \{a_1, \dots, a_n\}$, where each call $a_i = (f_i, \theta_i)$ names a tool $f_i \in \mathcal{F}$ and binds its arguments θ_i . When $n > 1$, the turn is a joint action. We score it as a set, as BFCL does [8]: the turn is correct if its calls can be matched one to one with the reference calls, whatever their order.

The agent’s decision core is a masked diffusion language model (dLLM). It writes A as a JSON array onto a canvas of mask tokens and fills the canvas over several denoising steps. To make every action well formed, the agent’s interface fixes the syntax of the turn in advance with a *skeleton*: the number of calls, the tool names, the argument names, and all brackets and quotes are written onto the canvas, and only the argument values are masked. Each value occupies a *slot* of ℓ mask tokens. The slots that hold the same argument in different calls of the same tool form a *sibling group*; these are the slots that can be confused with each other. Our skeleton is an oracle, built from the reference calls, so that every remaining error is an error in a value. Section 9 discusses what the oracle hides.

A slot need not be filled. A value ends when the model writes its closing token (the closing quote for a string; for other types, a comma or closing brace outside any bracket of the value), and the rest of the slot is filled with padding. Padding is forced and is not counted as a decision. The autoregressive (AR) reference agent uses the same skeleton and the same rule: it writes each value until its closing token and is cut off at the slot length, but it is never made to fill a slot. Surplus masks therefore cannot change its output; this is a property of the design, not a finding.

3.2 Decoding as a Protocol Among Slot-Level Deciders

At each denoising step, the dLLM predicts every masked position from the current canvas and commits some of them. A position committed in a step is chosen from its own predicted distribution given the canvas *before* that step, so the positions committed in the same step cannot see each other’s choices. We call the masked positions *slot-level deciders*. The decoding schedule then acts as a protocol that says which deciders move when:

- With k tokens per step in confidence order, the k most confident masked positions move together. With $k = 1$, one decider moves at a time and sees every earlier commitment; we call this *one token per step*. With $k \geq 2$, several deciders, possibly in different calls, move simultaneously.
- Left-to-right decoding ($k = 1$) fixes the order of moves instead of letting confidence choose it. A confidence threshold τ [12] lets every position whose top probability exceeds τ move at once.
- LLaDA2.0 decodes in blocks of 32 tokens, with bidirectional attention inside a block and causal attention across blocks; a block is finished before the next one starts. Slots in different blocks are never committed together, and the later slot sees the earlier one. In our skeletons only 3.1% (parallel) and

2.3% (parallel_multiple) of the pairs of sibling slots share a block, so LLaDA2.0’s calls take turns at any k .

3.3 Coordination Cues and Predictions

The sibling slots of a turn face a coordination problem. Each slot must take the value that belongs to its own call, for instance the i -th of several cities, and a slot moving in the same step as its siblings cannot see what they take. Under greedy decoding, deciders with near-identical views make identical choices. What can set them apart is a cue that every slot can read from the shared canvas and that tells each slot which value is its own. We consider three such cues:

- (1) *Mention order*. The request names the entities in some order, and each slot sees the position of its call on the canvas. Taking the i -th mentioned entity for the i -th call is a convention that all slots can follow without communicating, a focal point in the sense of Schelling [11].
- (2) *Slot position*. A slot’s position on the canvas is a role label. A label helps only if something orders the candidate values: “you are the second call” does not say which of all US cities is the second.
- (3) *Slot length*. Sibling values usually differ in length, so the number of masks in a slot can tell the slot which value it holds, provided the model reads the mask count as the length of the value.

These cues give three predictions, which Sections 4–7 test.

- P1** *Order is a focal point*. When the request orders its entities, the slots take the i -th entity for the i -th call at any k , so committing many tokens per step costs little, even when slot lengths cannot tell the calls apart.
- P2** *Symmetric calls collide*. When nothing tells the calls apart, slots committed in the same step take the same value, while slots that take turns do not. A team of LLM agents that act at the same time without seeing each other should collide in the same way, unless numbering the agents lets them follow a listed order.
- P3** *Length is a promise*. A slot reads its mask count as the length of its value. A slot of the wrong length therefore receives a value of the length it promises: a longer version of its own value, or a sibling’s value that fits. This happens at any k , including one token per step.

3.4 Models, Data, and Measures

Models. Dream [14] (Dream-v0-Instruct-7B) is a dLLM with full bidirectional attention, initialized from Qwen2.5-7B. LLaDA2.0 [2] (LLaDA2.0-mini, a mixture of experts with 16B parameters, 1.4B of them active per token) is a block-diffusion dLLM that supports tool calling. The AR reference agent is Qwen2.5-7B-Instruct [9], the instruction-tuned version of the base model that Dream is initialized from. All models see the same system prompt, which lists the tools as JSON and asks for a JSON array of calls, and all decode greedily.

Data. We use the `parallel` and `parallel_multiple` categories of BFCL v4 [8]: 400 requests with 2 to 8 calls (2.9 on average) and 3,063 value slots. LLaDA2.0’s experiments with wrong slot lengths use a subset of 100 requests, 50 evenly spaced in each category. For

Table 1: Committing more tokens per step, with exact slot lengths (BFCL parallel and parallel_multiple, 400 requests). Set accuracy and CCER in %; CCER (sym.) on the requests whose slot lengths cannot tell the calls apart (213 for Dream, 216 for LLaDA2.0); order: share of symmetric parallel requests whose i -th call took the i -th mentioned entity; fwd: forward passes per request. LTR: left to right, one token per step; τ : commit every position above confidence 0.9. Qwen2.5 (autoregressive) in the same skeleton: 86.5% set accuracy, 1.3% CCER.

	$k=1$	2	4	8	16	LTR	τ
<i>Dream (full attention)</i>							
Set acc.	89.8	88.0	87.0	86.3	84.5	89.3	89.0
CCER	1.3	1.3	1.5	1.8	2.0	1.0	1.3
CCER (sym.)	1.9	1.9	2.3	2.8	2.8	1.4	1.9
Order	95.2	95.2	95.0	96.1	96.1	97.5	95.1
Fwd	28.0	14.2	7.3	3.9	2.2	27.8	1.9
<i>LLaDA2.0 (32-token blocks)</i>							
Set acc.	87.8	87.5	86.5	83.3	79.8	–	–
CCER	2.5	2.8	3.3	4.0	4.5	–	–
CCER (sym.)	0.9	0.9	1.4	1.4	1.9	–	–
Order	95.1	96.3	96.3	95.1	93.8	–	–
Fwd	27.8	14.8	8.3	5.1	3.9	–	–

requests in which nothing tells the calls apart, we add 105 *choose-N* requests (Section 5).

Slot lengths. Exact: each slot has as many masks as its own reference value has tokens. *Surplus +s:* every slot gets $s \in \{1, 2, 4, 8\}$ extra masks. *Swap:* slot lengths are rotated by one position within each sibling group, on the requests where this changes at least one slot (165 for Dream, 34 for LLaDA2.0). *Estimate:* lengths predicted by the model itself in one forward pass, without reference values (Section 8).

Measures. Set accuracy is the BFCL verdict. The *cross-call error rate* (CCER) is the share of requests with at least one cross-call error, found by an optimal one-to-one matching of predicted and reference calls: a duplicated or missing call, a value that belongs to another call (cross-binding), a string spliced from two calls’ values (chimera), or an argument that the reference shares across calls but the prediction does not. At the slot level, we classify each value as the slot’s own reference value, a sibling’s value (*fit* if the sibling’s value has exactly the slot’s length), an *overflow* (the own value followed by more), a truncation of the own value, another value, or missing.

4 Ordered Requests: Same-Step Commits Coordinate on Mention Order

We first test **P1** on the BFCL requests with exact slot lengths (Table 1).

Committing many tokens per step costs little. Committing 16 tokens per step instead of one lowers Dream’s set accuracy from 89.8% to 84.5% and LLaDA2.0’s from 87.8% to 79.8%, while cutting the forward passes per request by factors of 13 and 7. The confidence

threshold, which lets every confident position move at once, keeps Dream at 89.0% with 1.9 forward passes per request, fewer than with $k = 16$. Cross-call errors stay rare at every k : at most 2.0% of requests for Dream and 4.5% for LLaDA2.0, against 1.3% for the AR agent. At the slot level, 95.4% of Dream’s slots hold their own value with one token per step and 92.0% with 16.

The calls are not told apart by their lengths. With exact lengths, sibling slots of different lengths could be told apart by their length alone, which would keep cross-call errors low without any coordination. On the requests in which every sibling group has slots of equal length (213 for Dream, 216 for LLaDA2.0), the cross-call error rate stays as low: from 1.9% to 2.8% for Dream and from 0.9% to 1.9% for LLaDA2.0 between $k = 1$ and $k = 16$.

Mention order is the focal point. Consider the length-symmetric requests of the parallel category, in which several calls of one tool differ only in their entities. There, the i -th call takes the i -th mentioned entity in 95.0–97.5% of Dream’s and 93.8–96.3% of LLaDA2.0’s requests, with no trend in k . Each slot can work out which entity is its own from the request and its own position, so it does not need to see what its siblings commit. Because BFCL lists the reference calls in mention order and our skeleton lays the calls out in that order, this convention is also the correct one.

Where the cost of parallelism lies. The cost falls mostly inside single values, not between calls. Dream gets 62 requests wrong at $k = 16$ against 41 at $k = 1$. Output that no longer parses grows from 4 to 21 requests: tokens of one value committed together that do not fit together. Requests with cross-call errors grow only from 5 to 8. LLaDA2.0’s calls take turns at any k , because its sibling slots almost never share a block (Section 3.2). Of its 32 additional failures, 18 are single-call value errors, 8 cross-call errors, and 6 output that does not parse.

When the request orders its entities, simultaneous moves need no communication: the order is a convention that every slot shares, as **P1** predicts.

5 Symmetric Requests: Same-Step Slots Collide When Nothing Orders the Choices

We now take the focal point away. A *choose-N* request asks for the weather in $n \in \{2, 3, 4\}$ cities without saying which call takes which city, either from a list (“...any three of these cities: Boston, Dallas, Toronto, Austin, Chicago and Paris”, $n + 3$ cities; 60 requests) or openly (“...three different US cities of your choice”; 45 requests). Any n different allowed cities are correct, so the calls must spread over the candidates, as players of a dispersion game must [3]. Every city slot holds one token, so each call makes a single decision. This token also limits which cities an open request can name, so for open requests we report only duplicates.

One canvas. Slots that move together collide (Table 2, top). With 16 tokens per step, all of Dream’s city slots move in the first step, and two calls take the same city in every open request and in 15.0% of list requests (10.0% at $k = 2$). With one token per step, no request has a duplicate; with sampling at $T = 0.7$, 0.4% of open runs do. Pair by pair at $k = 16$, 70.0% of the open pairs of slots and 5.5% of the list pairs take the same city. Collisions also propagate: at $k = 2$, each

Table 2: Symmetric requests (choose-N). Dup.: requests (%) in which two calls take the same city. Order: list requests (%) whose i -th call takes the i -th listed city. Top: one canvas holds all n calls. Below: n agents make one call each, at the same time or in turns (seeing the calls made so far), anonymous or numbered (“assistant i of n ”). Greedy decoding on 60 list and 45 open requests; $T=0.7$: five seeds (300 and 225 runs). Open requests hold one-token slots, so only duplicates are reported.

	Greedy		$T=0.7$		
	List		List		
	Dup.	Order	Dup.	Dup.	Dup.
<i>One canvas holds all n calls</i>					
Dream, $k=16$	15.0	11.7	100.0	17.3	100.0
Dream, $k=1$	0.0	48.3	0.0	0.0	0.4
LLaDA2.0, $k=16$	0.0	96.7	4.4	–	–
LLaDA2.0, $k=1$	0.0	100.0	0.0	–	–
Qwen2.5 (AR)	0.0	81.7	0.0	–	–
<i>A team of n Qwen2.5 agents, one call each</i>					
same time, anonymous	100.0	0.0	100.0	99.7	93.8
same time, numbered	100.0	0.0	97.8	100.0	93.8
turns, anonymous	0.0	91.7	0.0	–	–
turns, numbered	0.0	91.7	0.0	–	–
<i>A team of n Dream agents, one call each</i>					
same time, anonymous	100.0	0.0	100.0	87.7	100.0
same time, numbered	78.3	3.3	100.0	75.0	100.0
turns, anonymous	0.0	35.0	0.0	–	–
turns, numbered	0.0	26.7	0.0	–	–

of the 56 pairs that collide although they moved in different steps follows a same-step collision, the later slot repeating a city that two earlier slots already share. LLaDA2.0’s calls take turns block by block and rarely collide (at most 4.4% of open requests). Even the 76 pairs of its slots that share a block and move in the same step collide only twice, so block-wise turns are not the whole explanation. The AR agent writes its calls in turn and never collides.

Teams of agents. To test whether same-step slots act like agents that cannot see each other, we give the same requests to teams of n agents, each making one call through the same one-call skeleton (Table 2, bottom). Every agent receives the request followed by a note on the protocol: the agents act at the same time or in turns, seeing the calls made so far, and they are anonymous (“You are one of n assistants”) or numbered (“You are assistant i of n ”); the supplementary material gives the full texts. Anonymous agents that act at the same time receive identical inputs and, decoding greedily, make identical calls in every team, a sanity check. Agents that take turns never collide, with either model and in either variant. Agents that act at the same time collide whether or not they are numbered: numbered Qwen2.5 agents take the same city in every list team, and numbered Dream agents in 78.3% (55.0% with two agents, 95.0% with four). This contradicts our prediction that a number and a list would let agent i take the i -th listed city: only 3.3% of Dream’s numbered teams and none of Qwen2.5’s take the cities in list order. Instead the agents share a focal point; every Qwen2.5 agent that

acts at the same time takes the city listed first. Sampling does not break the symmetry. At $T = 0.7$, with a separate seed for every agent, teams that act at the same time still collide in 75.0–100% of list runs and 93.8–100% of open runs, and 99.8% of the anonymous Qwen2.5 agents still take the first city. LLMs tend to make the same choice as each other [1]: what pure coordination games reward [7], a dispersion game punishes.

What carries over. The protocol carries over from the agents to the slots. Deciders that move at the same time collide when nothing orders the choices (in 97.8–100% of open requests, slots and agents alike), and deciders that take turns do not. The role label does not carry over. A number in an agent’s prompt hardly separates simultaneous agents, whereas a slot’s position on the shared canvas separates same-step slots whenever the request lists the candidates: at $k = 16$, 94.5% of Dream’s list pairs take different cities although they move in the same step. Same-step slots are thus not anonymous agents but simultaneous deciders with a working role label, and they collide like agents that cannot talk when the label has nothing to map onto. **P2** holds for the protocol and fails for the agents’ labels.

6 Masks Are Length Promises

We now turn to the interface. Section 4 gave every slot exactly as many masks as its reference value has tokens. An agent in deployment cannot do this, because it does not know the values before it decodes them. The simplest departure gives every slot s extra masks; since the closing rule lets a value end anywhere in its slot, a model that knew where its values end would be unaffected.

One extra mask. The models are far from unaffected (Figure 2a,b). With one token per step, one extra mask per slot lowers Dream’s set accuracy from 89.8% to 15.5% and LLaDA2.0’s from 92.0% to 16.0%. The loss is not an artifact of scoring whole turns: the share of Dream’s slots that hold their own value falls from 95.4% to 34.0%. On the same scale, committing 16 tokens per step instead of one costs Dream 5.3 points of set accuracy and 3.4 points of slot accuracy; one extra mask costs 74.3 and 61.4. Parallel commits make matters worse, not better: with 16 tokens per step and one extra mask, Dream keeps 5.0% of its requests.

What fills the extra mask. Most wrong slots keep the promise that the mask count makes. At $s = 1$, 35.1% of Dream’s slots hold their own value followed by more, and only 0.5% hold a sibling’s value; the rest are other values (11.7%), truncations (0.4%), and output that does not parse (18.4%). In the request of Figure 1, one extra mask turns HSBC into “HSBC Bank” and the loan of \$500,000 into one of \$5,000,000; four extra masks make it “HSBC Holdings, Plc” and \$5,000,000,000. A numeric slot cannot be lengthened harmlessly: every extra digit multiplies the amount by ten.

Near misses hurt most. Set accuracy is not monotone in the surplus. For Dream it bottoms out at $s = 2$ (3.3%) and recovers to 60.0% at $s = 8$; LLaDA2.0 recovers to 38.0%. The probe in Figure 2c shows why. We write the reference value into the slot, leave s masks after it, and read the model’s distribution for the next position. With one or two masks left, Dream gives a closing token only 1.0% and 3.3% of its probability; it is nearly certain that the value goes on.

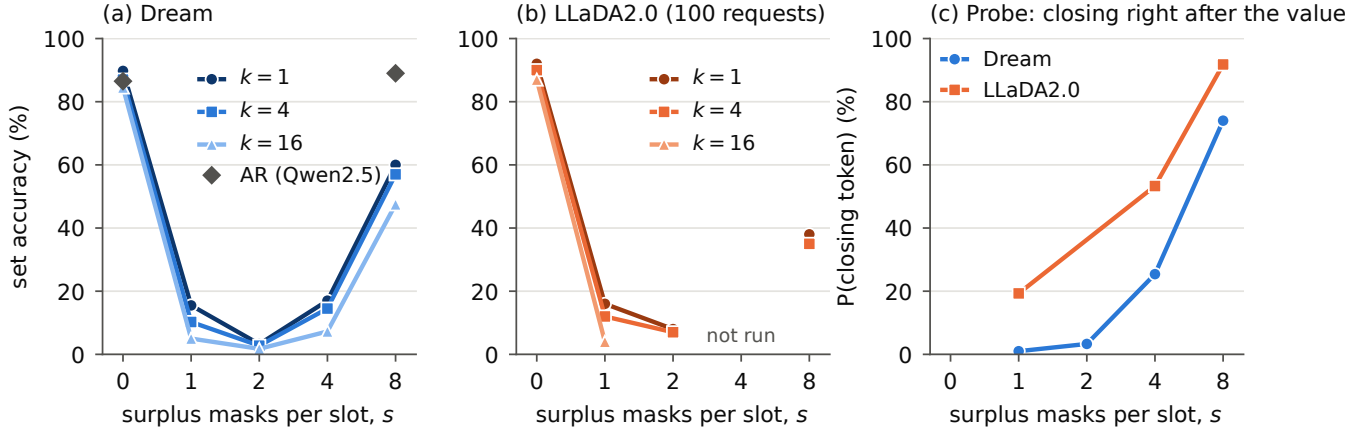


Figure 2: Every slot gets s masks more than its reference value has tokens. (a, b) Set accuracy against the surplus s , one line per number of tokens committed per step; LLaDA2.0 on its 100-request subset, where $s = 4$ was not run. The AR agent stops at the closing token and cannot be affected by surplus masks (Section 3.1). (c) Teacher-forced probe over all 3,063 slots: with the reference value written into the slot and s masks left, the mean probability that the next token closes the value.

With eight masks left, it gives 74.0%. LLaDA2.0 closes more readily (19.3% at $s = 1$, 91.8% at $s = 8$) but follows the same shape. A slot a little too long is read as a promise that the value is a little longer; a slot too long is read as room to spare.

Padding is not the way out. Our skeleton already lets the model write padding at any position of a slot, yet the models almost never pad inside a value: the median probability of padding right after the reference value is about 10^{-5} for Dream and below 10^{-5} for LLaDA2.0 at every s . S³ [13] reports the same over-generation when schema fields get ample masks, and prompts the model to fill unused fields with the word null. The closing token plays that role in our interface: it is the token that naturally ends a JSON value, and once it is written the rest of the slot is padded, so the model needs to take a single decision to stop. The probe measures exactly that decision. The AR agent, which stops at its closing token, keeps 89.0% set accuracy with eight extra masks per slot, as the design guarantees.

7 Length Errors Masquerade as Coordination Failures

The worry about parallel decoding predicts particular errors: a value bound to the wrong call, a duplicated call, a value spliced from two calls. This section shows that wrong slot lengths produce exactly these errors when the agent commits one token per step, where no two decisions are ever made at once.

Swapped lengths. We rotate the slot lengths within each sibling group by one position, so that every changed slot gets the length of a sibling’s value, and keep everything else fixed (Table 3). With one token per step, 79.4% of Dream’s swapped slots then hold the sibling value whose length fits the slot, against none of the same slots with exact lengths. The slot does not merely lack room for its own value: when the swapped slot is longer than the slot’s own value, so that the value would fit and the slot could close early, 79.1% still take the longer sibling (79.7% when it is shorter). Committing more tokens per step changes little (77.2% at $k = 4$, 73.3% at $k = 16$). LLaDA2.0

Table 3: Swapped slot lengths. Only the slots whose length the swap changes (514 slots in 165 requests for Dream, 120 in 34 for LLaDA2.0), next to the same slots with exact lengths. Slot shares in %: the slot’s own value, a sibling’s value whose length equals the slot’s, the own value followed by more, a prefix of it, anything else or nothing. Set accuracy and CCER in % of the same requests.

	Own	Sibling that fits	Over- fill	Trun- cated	Other	Set acc.	CCER
Dream, exact, $k=1$	96.3	0.0	0.2	0.0	3.5	93.3	0.6
swap, $k=1$	5.4	79.4	2.3	2.3	10.5	67.3	16.4
swap, $k=4$	6.2	77.2	2.3	2.3	11.9	64.8	19.4
swap, $k=16$	4.3	73.3	2.9	2.1	17.3	55.8	23.6
LLaDA2.0, exact, $k=1$	96.7	0.8	0.0	0.0	2.5	88.2	5.9
swap, $k=1$	12.5	47.5	6.7	13.3	20.0	17.6	44.1
swap, $k=4$	9.2	40.0	6.7	11.7	32.5	14.7	50.0

follows the length less often but still more often than its own value (47.5% against 12.5%, on 34 requests); the rest of its swapped slots are mostly truncated or other values. Set accuracy falls less than slot accuracy because a whole sibling group that rotates together can still match the reference set: in one request, Dream swapped both the artist and the play time of two calls, which is a correct set of calls in another order.

The length outranks the order. A swap sets two cues against each other. The mention order points each slot to its own value, which is what the models follow when the cues agree (Section 4). The length points to a sibling’s value. When they disagree, Dream follows the length about 15 times as often as the order (79.4% against 5.4%), and LLaDA2.0 about 4 times as often. In coordination terms, a signal written on the shared canvas overrides the focal point offered by the request.

Table 4: What an evaluator sees. Requests (%) with each kind of cross-call error, when the agent commits 16 tokens per step with exact slot lengths (shaded) and when it commits one token per step with wrong slot lengths, on the same requests. Estimate: lengths predicted by the model in one forward pass (Section 8). Dup.: a duplicated call; bound: a value that belongs to another call; chim.: a string spliced from two calls’ values; shared: an argument that the reference shares across calls and the prediction does not.

Slot lengths	n	Dup.	Bound	Chim.	Shared	CCER
<i>Dream</i>						
exact, $k=16$	400	0.5	0.8	0.0	0.8	2.0
estimate, $k=1$	400	1.5	3.5	1.0	2.3	7.5
+1, $k=1$	400	0.5	0.8	3.0	1.5	5.8
+8, $k=1$	400	1.0	0.5	0.0	0.3	1.8
exact, $k=16$	165	0.0	0.6	0.0	0.6	1.2
swap, $k=1$	165	0.0	15.2	1.8	0.0	16.4
<i>LLaDA2.0</i>						
exact, $k=16$	100	1.0	1.0	0.0	1.0	3.0
estimate, $k=1$	100	0.0	6.0	2.0	7.0	14.0
+1, $k=1$	100	2.0	1.0	7.0	4.0	12.0
+8, $k=1$	100	11.0	26.0	1.0	3.0	36.0
exact, $k=16$	34	2.9	2.9	0.0	2.9	8.8
swap, $k=1$	34	0.0	29.4	17.6	0.0	44.1

What an evaluator sees. Table 4 counts the cross-call errors of our taxonomy under two conditions on the same requests: the most parallel decoding with exact lengths, and one token per step with wrong lengths. In the setting closest to deployment, where the model predicts its own slot lengths (Section 8), one-token-per-step decoding has cross-call errors in 7.5% of Dream’s requests and 14.0% of LLaDA2.0’s, against 2.0% and 3.0% at 16 tokens per step with exact lengths. Chimera values, which splice individually likely tokens of two values into one (ParallelBench’s “New City” from “New York” and “Mexico City”, which it attributes to parallel decoding [4]), appear one token per step: in 3.0% of Dream’s requests with one extra mask and 7.0% of LLaDA2.0’s, against none at $k = 16$. LLaDA2.0 writes “HSBC Fargo” into the first call of Figure 1 with one extra mask. With eight extra masks, LLaDA2.0 duplicates a call in 11.0% of its requests and binds a value to the wrong call in 26.0%. One condition does not follow the pattern: Dream with eight extra masks has as few cross-call errors as with exact lengths (1.8% against 2.0%).

What the wrong actions would do. If the agent executed its calls, wrong lengths would change what it does, not only how it is scored. With one extra mask, 33.3% of Dream’s turns would pass a number that is off by a factor of ten or more, against none with exact lengths. With swapped lengths, 13.9% of the affected turns would act on another call’s entity, against 1.0% of all turns at $k = 16$ with exact lengths. With its own length estimates and one token per step, Dream would pass a number off by a factor of ten in 19.8% of its turns and act on another call’s entity in 4.0%.

Table 5: Coping with unknown value lengths. (a) A logit bias β toward the closing and padding tokens of every slot, Dream at $k=4$ with s surplus masks per slot (exact lengths: 87.0% set accuracy). Parses: share of output that parses. (b) Slot lengths that the model predicts in one forward pass: share of slots whose estimate is exact, too long, or too short, and set accuracy when the agent decodes with the estimates, next to exact lengths on the same requests (shaded). All values in %.

(a) Closing bias, Dream, $k=4$				
β	0	2	4	8
Set acc., $s=2$	2.8	2.8	6.5	13.8
Set acc., $s=8$	57.0	72.3	71.3	20.3
Parses, $s=8$	89.0	93.3	88.3	37.3

(b) One-forward length estimate						
	Slots			Set acc.		
	Exact	Long	Short	$k=1$	4	16
Dream (400)	51.6	44.2	4.1	42.3	39.0	32.3
exact lengths	–	–	–	89.8	87.0	84.5
LLaDA2.0 (100)	72.8	12.6	14.7	39.0	38.0	–
exact lengths	–	–	–	92.0	90.0	–

An evaluation protocol. An error that persists when the agent commits one token per step is not an error of parallel decoding. Before attributing a dLLM agent’s cross-call errors to parallel commits, an evaluator should decode the same requests with one token per step and with controlled slot lengths (exact, swapped, and surplus), and attribute to parallelism only the difference that remains.

8 Designing and Evaluating dLLM Agents

An agent in deployment must give every slot a length before it knows the value. We tested two ways to cope with this (Table 5).

A bias toward closing. Adding β to the logits of the closing and padding tokens in every slot rescues large surpluses: with eight extra masks per slot, Dream’s set accuracy at $k = 4$ rises from 57.0% to 72.3% at $\beta = 2$, still below the 87.0% it reaches with exact lengths. It does not rescue near misses: with two extra masks, set accuracy stays at 2.8% at $\beta = 2$ and reaches only 13.8% at $\beta = 8$, a bias that ruins the large surplus: many slots close at their first position and stay empty, and with eight extra masks only 37.3% of the output parses. No single bias serves both cases. This matches the probe of Section 6: right after the value, the model is far less willing to close with one or two masks left than with eight, so a bias strong enough to close the near misses also closes many slots before their value starts.

Lengths from the model. The agent can also ask the model. We give every slot as many masks as its type allows, run one forward pass, read the probability of a closing token at each position of a slot as the hazard that its value ends there, and take the most likely length. CAL [5] searches for the length of a single infilled span with 11–18 extra forward passes and leaves several spans to future work; its search may well be more accurate than our estimate, but we did not run it. Our estimate is exact for 51.6% of Dream’s slots and 72.8% of LLaDA2.0’s, but all of a request’s slots are exact in

only 14.5% and 27.0% of the requests. With these lengths and one token per step, Dream reaches 42.3% set accuracy and LLaDA2.0 39.0%, against 89.8% and 92.0% with exact lengths, and committing more tokens per step costs more than with exact lengths (32.3% for Dream at $k = 16$). The two models miss differently. Dream’s estimates are mostly too long, and by a median of 7 tokens; only 4.8% of its slots are one or two tokens too long. This is the regime in which a closing bias helps, but we did not run the two together. LLaDA2.0’s estimates are about as often too short (14.7%) as too long (12.6%), and a slot that is too short cannot hold its value at all.

Recommendations. The interface writes the signals that the slot-level deciders read, the protocol decides who moves when, and the evaluation decides what is blamed. (1) *Interface: avoid near misses.* A slot one or two tokens too long is the worst choice and a slot too short cannot be recovered. An agent that cannot know its lengths should leave generous room and bias the slots toward closing; with eight extra masks and $\beta = 2$ this reached 72.3%, against 87.0% with exact lengths. (2) *Protocol: let ordered calls move together and symmetric calls take turns.* When the request orders the calls, parallel commits are nearly free: the confidence threshold keeps Dream at 89.0% with 1.9 forward passes per request (Section 4). When nothing tells the calls apart, same-step commits collide, one token per step avoids the collisions, and block-wise decoding nearly does; sampling does not, for slots or for teams of agents (Section 5). (3) *Evaluation: control the lengths first.* Only the difference between parallel commits and one token per step, under the same controlled lengths, is a cost of parallelism (Section 7).

9 Discussion, Limitations, and Conclusion

Beyond one agent. Nothing in the analysis of Section 3 depends on diffusion. Whenever several decisions are taken at once by deciders that cannot see each other’s choices, the three cases of this paper apply. A convention that all deciders share, such as the order in which a request names its entities, assigns the roles and makes simultaneous moves safe. Symmetric roles collide unless the deciders take turns. And a shared feature that the designer did not intend as a signal, such as the length of a slot, is read and obeyed, so a wrong value of it mis-binds even decisions taken one at a time. Multi-agent LLM systems in which several workers claim subtasks of a shared plan at the same time [6, 10] face the same three cases, with the format of the plan in the place of the skeleton. Section 5 shows the second case in teams of LLM agents: agents that act at once collide on a shared focal point whether they are numbered or sample, and agents that take turns never collide. Such systems need turn-taking, or a shared order that assigns the roles, rather than role names in the prompt.

Limitations. The oracle skeleton fixes the number of calls, their tools, and their argument names, so our numbers isolate the values and do not describe a deployed agent: without the skeleton, Dream reaches 75.3% set accuracy at $k = 2$, against 88.0% with it. Exact lengths also tell the model which value goes where; the length-symmetric requests (Section 4) and the swap (Section 7) control for this, but no skeleton removes it. We tested two dLLMs with greedy decoding only, and ran LLaDA2.0 on subsets of 100 requests (surplus and estimated lengths) and 34 requests (swap), where differences of

a few points are within noise **TBD(confidence intervals)**. The open choose-N variant gives each city one token, which limits which cities a slot can write, so we report only its duplicates. We did not run a prompted placeholder baseline in the style of S^3 [13], the length search of CAL [5], or the closing bias together with estimated lengths. Finally, the skeleton tokenizes the fixed text around each slot on its own, so its token boundaries can differ from those of free generation.

Conclusion. A dLLM agent that issues several tool calls in one turn coordinates them the way players who cannot talk do. When the request orders its entities, the order is a focal point, and committing many tokens per step costs little. When nothing tells the calls apart, same-step commits collide, and taking turns avoids the collisions. The most costly failures come from the action interface: the agent reads the number of masks in a slot as a promise about the length of its value and keeps the promise, so a slot of the wrong length gets a value of the wrong size or a sibling’s value, even with one token per step. These errors look like the failures blamed on parallel decoding. Designers of dLLM agents should get slot lengths right before they give up parallelism, and evaluators should control slot lengths before they blame it.

Use of AI Tools

We used Claude Opus 5.5 (Anthropic) as an assistant throughout this project: to help design the experiments, to write and run the experiment and analysis code, to analyse the results, and to draft and revise the text. The prompts used to run the experiments are included in the supplementary material, and every number in the paper is produced by the released scripts from the released model outputs.

References

- [1] Gonzalo Ballester, Hadi Hosseini, Samarth Khanna, and Ran I. Shorrer. 2026. Strategic Algorithmic Monoculture: Experimental Evidence from Coordination Games. *arXiv preprint arXiv:2604.09502* (2026).
- [2] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, et al. 2025. LLaDA2.0: Scaling Up Diffusion Language Models to 100B. *arXiv preprint arXiv:2512.15745* (2025).
- [3] Trond Grenager, Rob Powers, and Yoav Shoham. 2002. Dispersion Games: General Definitions and Some Specific Learning Results. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2002)*. 398–403. doi:10.5555/777092.777156
- [4] Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjae Lee, Yuchen Zeng, Shuibai Zhang, Coleman Hooper, Yuezhou Hu, et al. 2026. ParallelBench: Understanding the Trade-offs of Parallel Decoding in Diffusion LLMs. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=OsZr5T7Cd0>
- [5] Hengchang Liu, Zhao Yang, and Bing Su. 2026. Diffusion LMs Can Approximate Optimal Infilling Lengths Implicitly. *arXiv preprint arXiv:2602.00476* (2026).
- [6] Yuzhen Mao, Jerry Gu, Aadi Chauhan, Qizheng Zhang, Hangoo Kang, and Azalia Mirhoseini. 2026. Decentralized Multi-Agent Systems with Shared Context. *arXiv preprint arXiv:2606.10662* (2026).
- [7] Judith Mehta, Chris Starmer, and Robert Sugden. 1994. The Nature of Salience: An Experimental Investigation of Pure Coordination Games. *American Economic Review* 84, 3 (1994), 658–673.
- [8] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 48371–48392. <https://proceedings.mlr.press/v267/patil25a.html>
- [9] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

- [10] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitsin, Denis Kuznedelev, and Dan Alistarh. 2025. Hog-wild! Inference: Parallel LLM Generation via Concurrent Attention. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1551
- [11] Thomas C. Schelling. 1960. *The Strategy of Conflict*. Harvard University Press, Cambridge, MA.
- [12] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2026. Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=3Z3Is6hnOT>
- [13] Zhen Xiong, Yujun Cai, Zhecheng Li, and Yiwei Wang. 2026. Unveiling the Potential of Diffusion Large Language Model in Controllable Generation. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=qhd0qv6L0k>
- [14] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7B: Diffusion Large Language Models. *arXiv preprint arXiv:2508.15487* (2025).